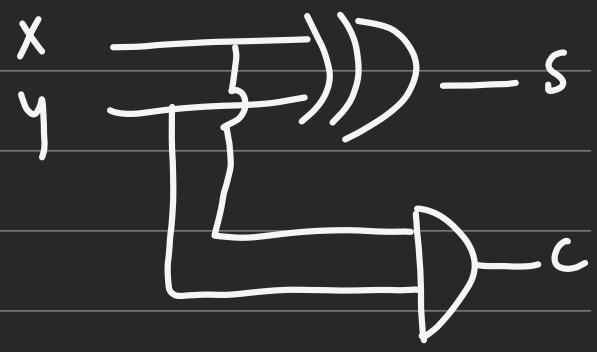
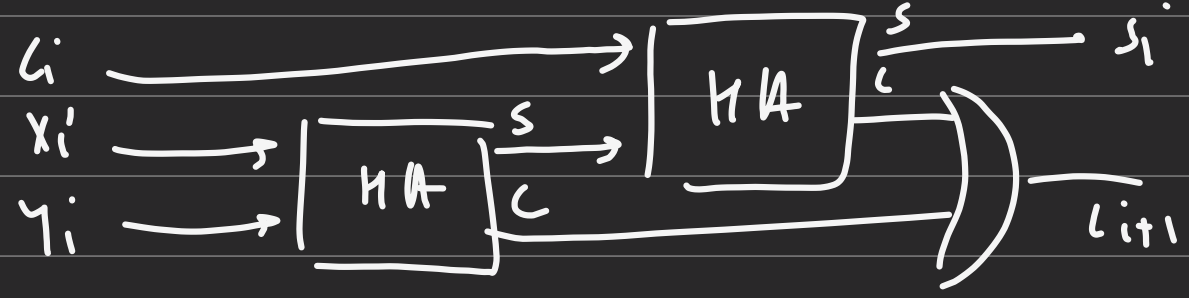
Adders

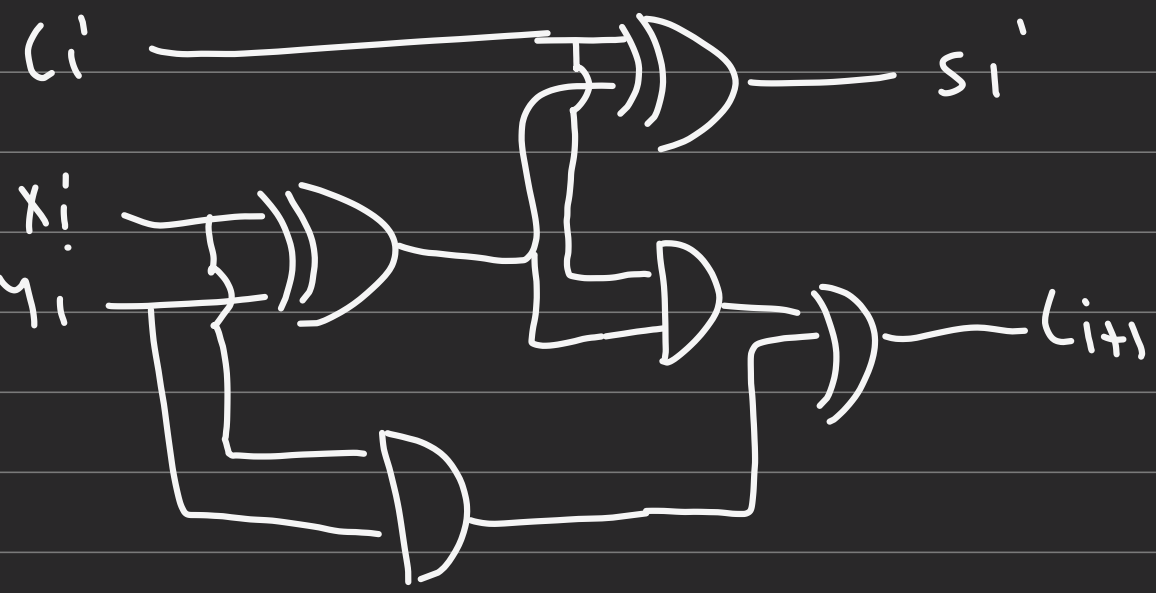
① Half Adder

$$S = x \oplus y$$
$$C = xy$$



② Full Adder

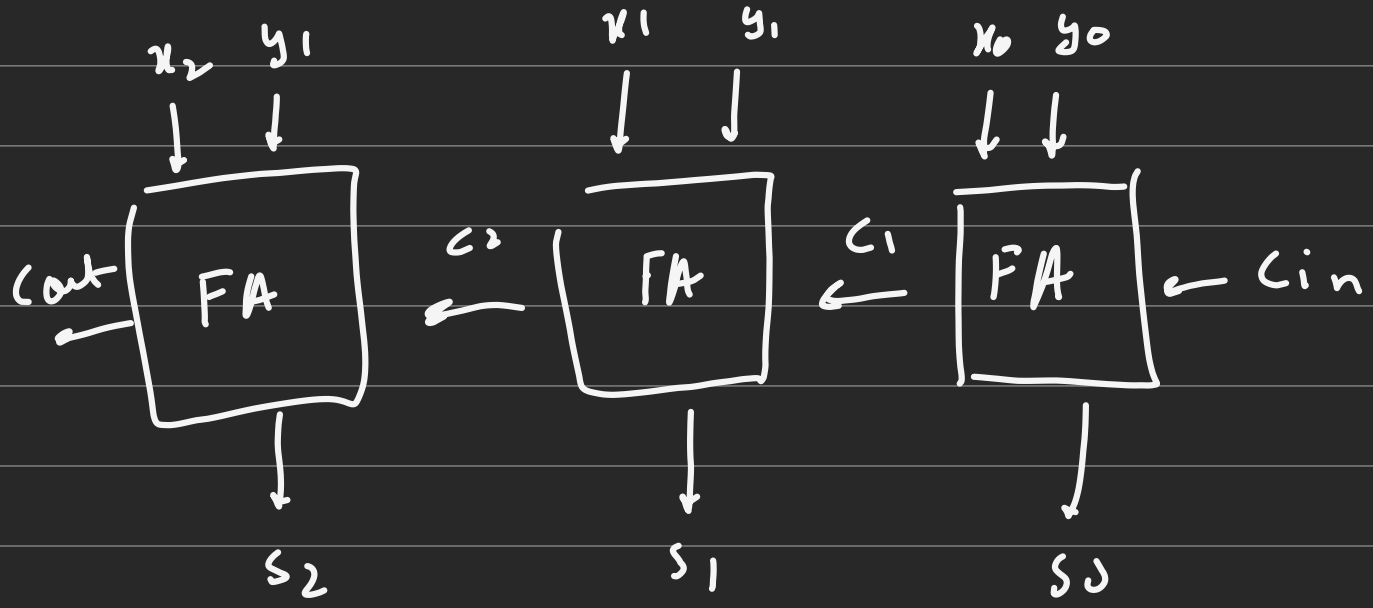




$$S = x_i \oplus y_i \oplus C_i$$

$$C = (x_i \oplus y_i)c_i + x_i y_i$$

③ Ripple Carry Adder



input [3:0] X, Y

→ least significant

→ most significant

④ n bit RCA

(parameter n = 4)

→ will take this as default value

(input ____

____)

assign c[0] = ____

: c[n]

only when using int. inside gen loop.

```
generate
  genvar k
  for (k=0; k<n; k=k+1)
  begin : stage
    |
  end
endgenerate
```

can't use k++

to give name of each instance

Subtractor

→ 2's complement

@ first 1's complement (flip all bits)

@ Add 1

ie

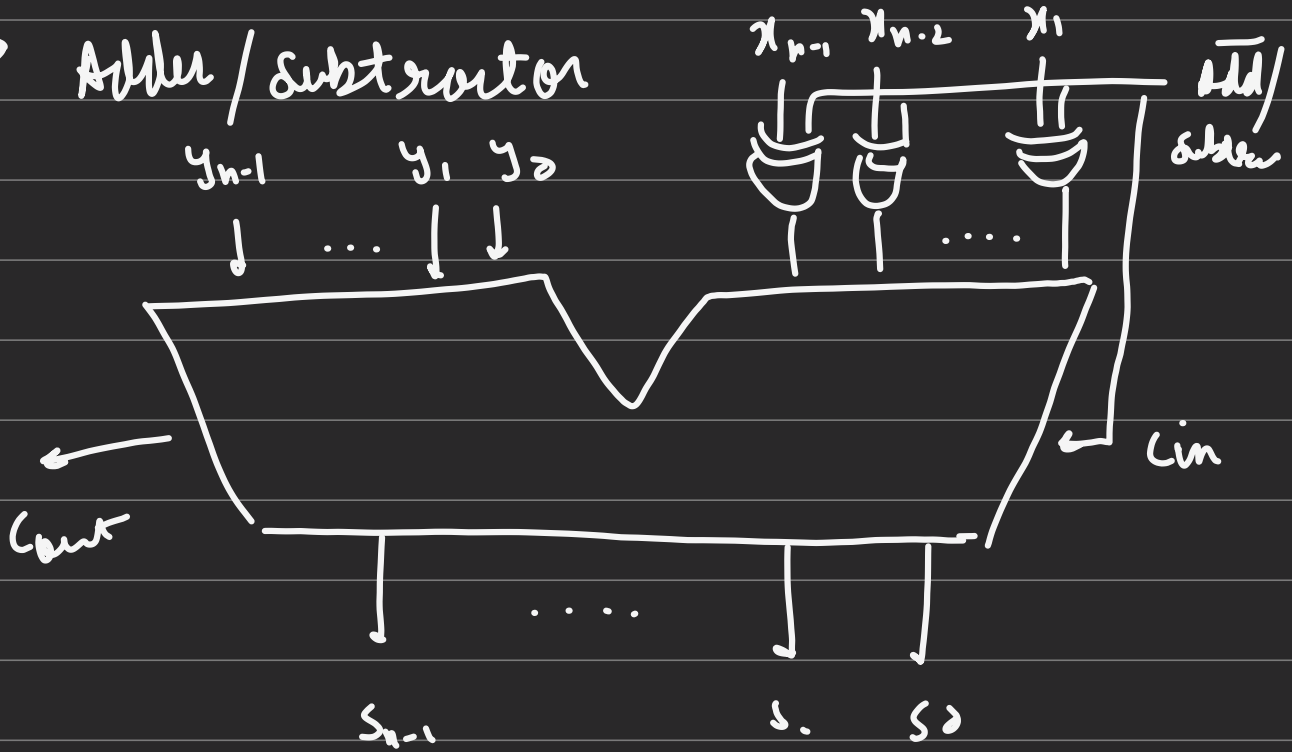
```
  1010
  0101   ← 1's complement
+    1
  ----
  0110   ← 2's complement
```

↳ So subtraction will be adding 2's complement of it.

i.e

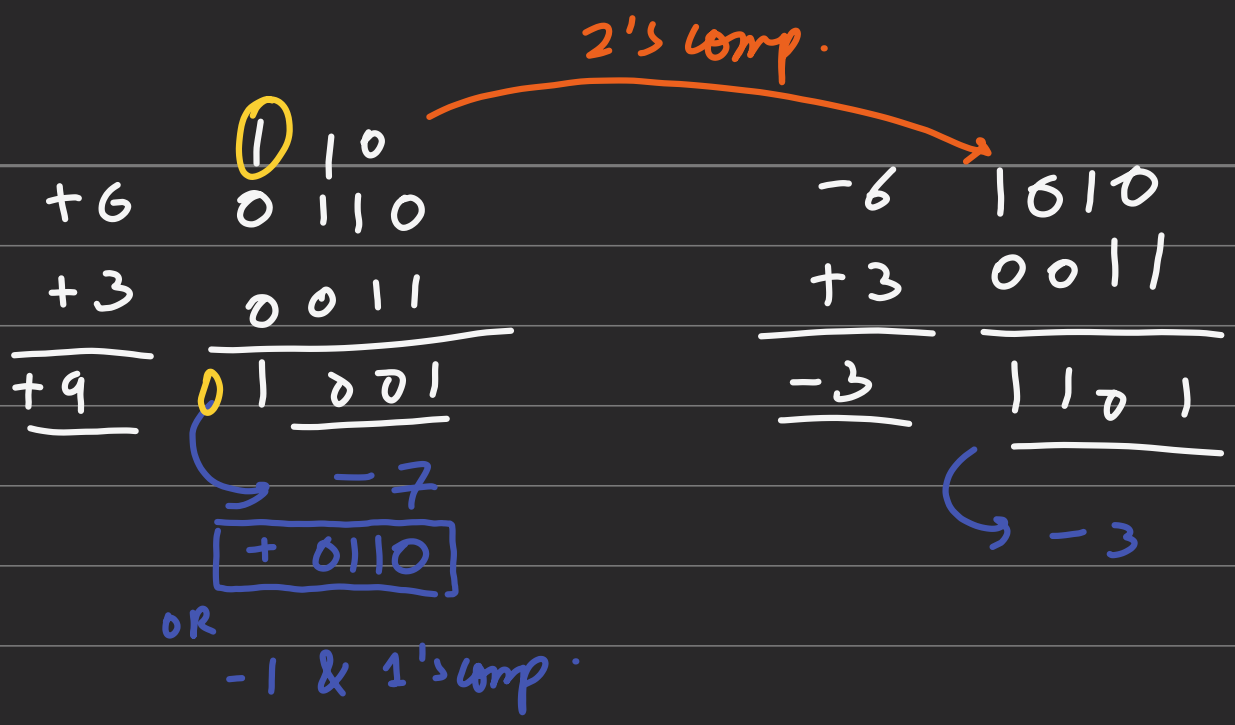
14	1110	1110
− 10	1010 →	+ 0110
4	0100	← 10100

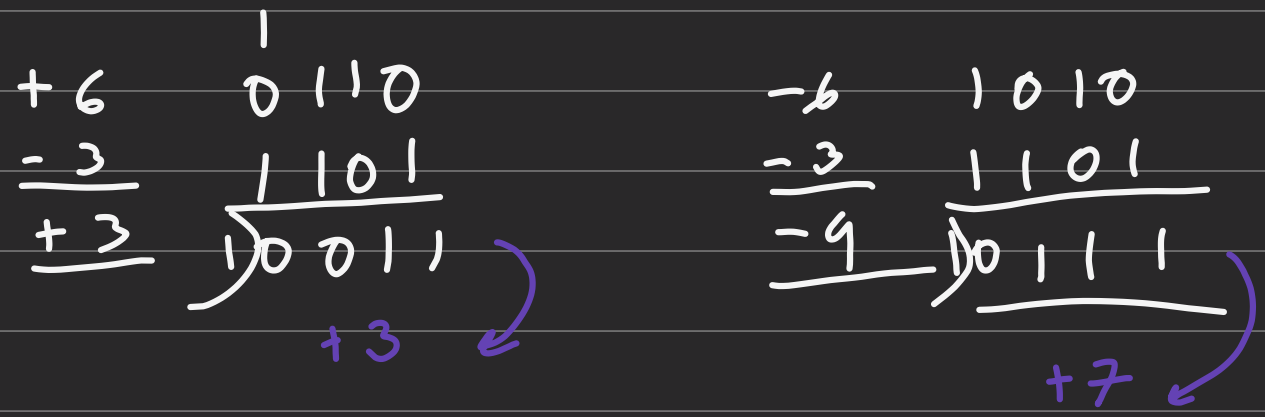
↳ Adder / Subtractor



Signed no. & Arithmetic overflow

4 bit no. (−8 to 7)





way to check overflow ..

if $c_{in} \wedge c_{out}$ of last bit is 1 then overflow.

M2 if MSB are 1 & output is 0 or reverse bit.

5 bit

4 bit

→ assign sum = x + y

↳ so + operator will automatically extend x to 5 bits & then add, so as per mem length

Test bench

module ——— ();

1) Declare local reg & wire identifiers
2) Instantiate the module under test.
3) Specify a stopwatch to stop the simulation
4) Generate stimuli, using initial and always
5) Display the output response (text or graphics)

endmodule

i.e

module ——— ();

parameter n = 4;

reg

wire

;

adder subtractor #(.n(n)) uut (

)

initial

#40 $finish

initial

begin

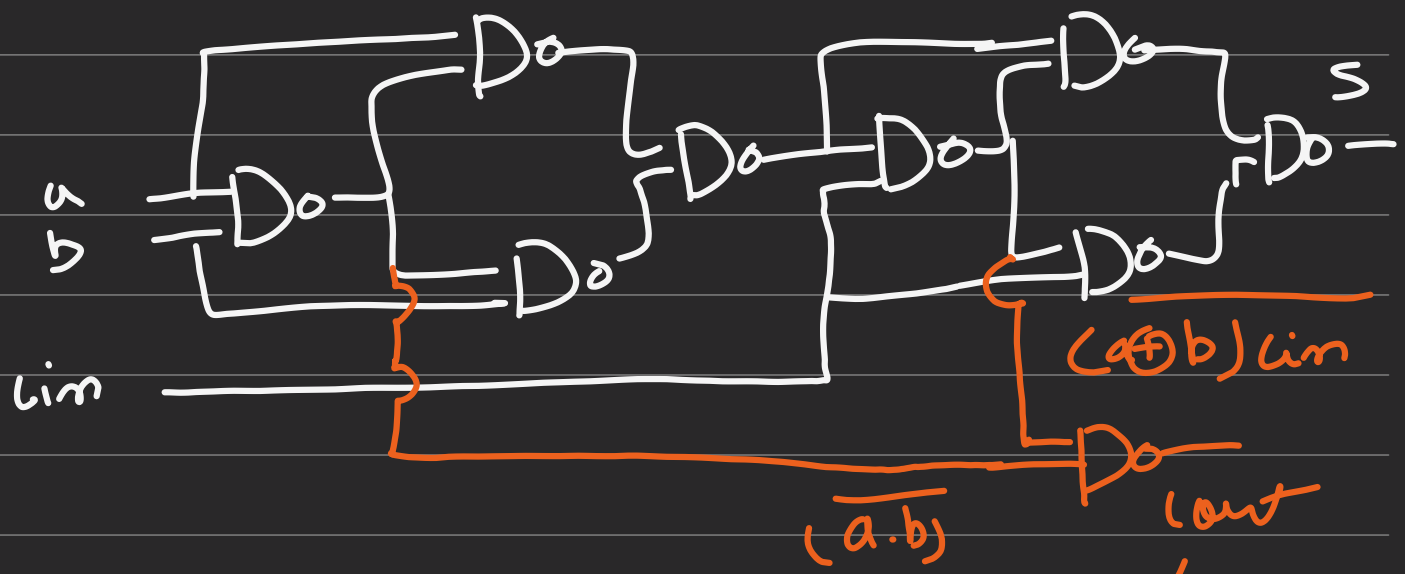
#10

#10

end

endmodule

Full Adder using NAND gates.



$$\overline{\left(\overline{(a\cdot b)}\cdot\overline{((a\oplus b)c_{in})}\right)}$$

$$= (a\cdot b) + (a\oplus b)c_{in}$$